智阅架小程序模块

一、实验目的

1. 掌握微信小程序开发工具的基本使用方法
2. 学习通过华为云 IoT 平台 API 获取设备影子数据
3. 实现设备状态（温度、光照、RGB 灯）的实时展示
4. 实现 LED 状态指示灯动画效果（常亮/闪烁）
5. 掌握小程序页面的布局与样式设计

二、实验环境

项目	配置
开发工具	微信开发者工具 Stable 版
操作系统	Windows 11
小程序基础库	3.6.6
云平台	华为云 IoTDA
通信协议	HTTPS / MQTT
测试设备	ESP32 开发板

三、实验原理

3.1 系统架构

小程序作为展示层，通过 HTTPS 请求调用华为云 IoT 平台的 REST API，获取 ESP32 设备上报的设备影子数据。设备影子是华为云为每个设备

存储的 JSON 文档，包含设备最新上报的状态信息（温度、光照、RGB 灯值等）。

3.2 数据流向

ESP32 采集传感器数据 → MQTT 协议上报华为云 → 设备影子更新 → 小程序调用 API → 页面可视化展示

3.3 关键技术

- 设备影子：存储设备最新状态，小程序无需直接连接设备
- Token 认证：通过 IAM 服务获取临时令牌（有效期 24 小时），用于调用华为云 API
- 轮询刷新：每 5 秒自动获取最新数据，实现准实时展示
- CSS 动画：实现 LED 指示灯常亮、单次闪烁、慢闪、快闪等效果

四、实验内容与步骤

4.1 小程序项目创建

在微信开发者工具中新建项目，命名为 smart_bookshelf，选择 JavaScript 基础模板，不使用云服务。创建完成后，新建 book_shelf 页面作为主页面。

4.2 页面功能设计

页面从上到下划分为六个功能区域：

1. 顶部状态栏：显示系统名称、设备在线状态、最后更新时间
2. 环境监测区：以卡片形式展示温度和光照强度，包含数值和文字提示
3. LED 状态指示灯区：展示五个状态灯的实时状态（系统待机、图

书归还、图书借出、网络连接、温度告警）

4. 藏书统计区：展示总藏书量、在架数量、借出数量

5. 最近动态区：以时间线形式展示图书借还记录

6. 调试日志区：可折叠的控制台日志，便于问题排查

4.3 华为云 API 对接

小程序需要调用两个华为云接口：

- Token 获取接口：使用账号密码进行身份认证，获取临时令牌
- 设备影子查询接口：使用令牌调用，获取指定设备的影子数据

在页面加载时自动执行 Token 获取，成功后启动每 5 秒自动刷新的定时器，下拉页面也可手动刷新。

4.4 LED 动画效果设计

状态	动画效果	触发条件
系统待机	绿灯常亮	设备在线
图书归还	蓝灯闪一下	检测到 returned 事件
图书借出	红灯闪一下	检测到 taken 事件
网络异常	红灯慢闪（1 秒周期）	Token 失效或请求失败
温度告警	红灯快闪（0.5 秒周期）	温度>30° C

4.5 页面布局与样式

采用 Flex 布局，卡片式设计，圆角边框。颜色方案以白色卡片搭配

浅色渐变背景，头部使用紫色渐变增强视觉效果。

五、遇到的问题及解决方案

问题	解决方案
Token 过期（24 小时）后请求返回 401	在每次 API 请求失败时检测状态码，若为 401 则自动重新获取 Token 并重试请求
小程序 WXML 模板不支持除法运算	在 JavaScript 层预先计算好百分比值，存入 data 中，WXML 直接引用
图片资源缺失	将图片引用改为文字图标，避免依赖外部图片资源
设备影子数据结构不熟悉	查阅华为云官方文档，明确数据路径为 res.data.shadow[0].reported.properties

ESP32 调用 DeepSeek AI 模块

一、实验目的

1. 掌握 ESP32 通过 HTTPS 协议调用云端 AI API 的基本方法
2. 实现 ESP32 与 DeepSeek 大语言模型的对接

3. 将 AI 功能集成到智能图书管理系统中，实现图书推荐、智能问答等功能
4. 理解边缘设备调用云端 AI 的架构模式

二、实验环境

2.1 硬件环境

设备	型号	用途
开发板	ESP32 Dev Module	主控，运行 AI 调用程序
传感器	NFC RC522	检测图书借还
传感器	条码扫描模块	扫描图书 ISBN

2.2 软件环境

软件	版本	用途
Arduino IDE	2.0+	代码开发与烧录
ESP32 开发板库	2.0.14+	硬件支持
ArduinoJson	6.x	JSON 解析
HTTPClient	-	HTTPS 请求

2.3 网络环境

项目	配置
WiFi	2.4GHz 网络
API 地址	https://api.deepseek.com/v1/chat/completions
认证方式	Bearer Token
端口	443 (HTTPS)

三、实验原理

3.1 系统架构

采用设备直连云端 API 的架构模式，ESP32 直接通过 HTTPS 协议调用 DeepSeek API，无需中间服务器。

3.2 DeepSeek API 简介

DeepSeek 提供兼容 OpenAI 格式的 API 接口，主要参数：

参数	类型	说明
model	String	模型名称，使用 deepseek-chat
messages	Array	对话消息数组，包含 role 和 content
temperature	Float	控制随机性，范围 0-1
max_tokens	Int	最大输出 token 数

3.3 HTTPS 通信流程

1. ESP32 建立 TLS 连接
2. 发送 HTTP POST 请求，携带 JSON 格式的请求体
3. 接收 DeepSeek API 返回的 JSON 响应
4. 解析响应提取 AI 回复内容

四、实验内容与步骤

4.1 获取 DeepSeek API Key

1. 访问 DeepSeek 开放平台（https://platform.deepseek.com/）
2. 注册/登录账号
3. 进入 API Keys 页面，点击"Create API Key"
4. 复制生成的 API Key（格式：sk-xxxxxxxxxxxxxx）

4.2 核心代码实现

添加头文件：

```
cpp
#include <WiFiClientSecure.h>  // HTTPS 支持
#include <HTTPClient.h>        // HTTP 请求
#include <ArduinoJson.h>       // JSON 解析
```

配置 API 参数：

```
cpp
const char* DEEPSEEK_API_KEY = "sk-xxxxxxxxxxxxxxxx";
const char* DEEPSEEK_API_URL = "https://api.deepseek.com/v1/chat/completions";
```

AI 调用函数核心逻辑：

- 检查 WiFi 连接状态
- 配置 HTTPS 客户端（开发阶段跳过证书验证）
- 构建 JSON 请求体（包含 model、messages、temperature、max_tokens）
- 发送 POST 请求并处理响应

- 解析返回的 JSON，提取 AI 回复内容

4.3 集成到图书管理系统

在借还书事件中自动触发 AI 推荐：还书成功后，根据归还的图书标题自动生成推荐问题（如“推荐一本类似《XXX》的书”）。

五、实验结果

5.1 功能验证

测试场景	输入	AI 响应示例	结果
图书推荐	″推荐一本类似《三体》的书″	″《球状闪电》- 同作者刘慈欣的科幻佳作″	通过
基础问答	″你好，介绍一下自己″	″我是 DeepSeek，AI 助手，随时为您服务″	通过
环境联动	″今天很热，适合读什么书″	″推荐《夏日终曲》，温暖治愈″	通过

5.2 性能指标

测试项	数值
WiFi 连接时间	约 3-5 秒
API 响应时间	约 2-6 秒

测试项	数值
单次请求内存消耗	约 4-6KB

六、遇到的问题及解决方案

问题	现象	解决方案
HTTPS 连接失败	HTTPClient 返回-1 错误	开发阶段使用 client.setInsecure()跳过证书验证
JSON 解析失败	deserializeJson 返回错误	增大 JSON 缓冲区至 4096 字节，或使用 DynamicJsonDocument
API 返回 401	认证失败	检查 API Key 格式（应以 sk-开头），重新生成 Key
API 返回 402	账户余额不足	登录平台充值或使用免费试用额度

实验总结

本实验成功实现了两个核心模块：

1. 小程序展示模块： 完成了与华为云 IoT 平台的数据对接，实现了设备状态的实时展示和 LED 动画效果。小程序能够每 5 秒自动获取 ESP32 上报的温度、光照、RGB 灯值等数据，并通过 LED 动画效果模拟了实际三色灯的工作状态。掌握了小程序开发流程、云 API 调用方

法、CSS 动画实现等技能。

2. AI 调用模块： 实现了 ESP32 直接调用 DeepSeek API，完成了图书智能推荐功能。验证了边缘设备调用云端 AI 的可行性，HTTPS 通信是 ESP32 调用云端 API 的关键，需要正确配置 TLS；内存管理在嵌入式设备中尤为重要，JSON 缓冲区大小需要合理设置；系统提示词的设计直接影响 AI 回答质量。该方案适合资源受限的物联网设备，后续可添加流式响应、常见问题缓存、语音交互等改进。